Aegis: Searching for and Defending Against Harmful Ontology Edits in Ontology-Grounded Retrieval

A white paper

Lam Nguyen
Case Western Reserve University
Cleveland, Ohio, USA
ltn18@case.edu

Redad Mehdi
Case Western Reserve University
Cleveland, Ohio, USA
mxm1684@case.edu

Roger H. French
Case Western Reserve University
Cleveland, Ohio, USA
roger.french@case.edu

Yinghui Wu
Case Western Reserve University
Cleveland, Ohio, USA
yinghui.wu2@case.edu

Abstract

Retrieval over a knowledge graph is shaped by its ontology: the ontology decides which nodes match a query, which edges may be followed, and what an answer means. An attacker who edits a few ontology triples, and no graph facts, can therefore change what an agent retrieves and answers. Measuring how a given ontology defect degrades a retrieval pipeline is a first step; this white paper asks the two adversarial questions that follow. (1) *Attack*: given a query workload, a library of edit operators, and a budget of k actions, which chain of edits does the most harm? We model attack construction as planning over a tree of action sequences, show that harm is not submodular in the edits, so greedy search carries no guarantee, and search the tree with a bandit at every node (UCT). (2) *Protection*: given a budget of M triples the defender may lock, which locks minimize the harm of the attacker's best response? We cast this as a leader–follower game and lock triples one at a time against the attacker's current best chain. Aegis implements both on a transactional ontology store that logs every edit with its undo, measures each edit's affected area over the workload, detects the first harmful commit, and rolls back to the last healthy state. On four real ontologies (three from materials science and the 2,743-class mouse anatomy ontology), the bandit search finds chains up to 64% more harmful than greedy and beam search at the same number of harm evaluations on one ontology, ties beam on the others at two actions, and trails it at longer chains; with more evaluations it converges where random search does not. Best-response locking is never worse than locking the highest-impact triples and leaves 0.133 residual harm where they leave 0.400; six to ten locks remove nearly all of a two-action attacker's harm on every ontology. Every number here is measured; claims we have not yet tested are marked as planned.

Keywords

ontology, knowledge graphs, GraphRAG, adversarial attacks, Monte Carlo tree search, bandits, Stackelberg games, transactions

1 Introduction

Language-model agents that answer questions over a knowledge graph (KG) retrieve evidence before they reason. In GraphRAG and KG-RAG systems [3] the retrieval step is guided by an ontology: the ontology says that Annealing is a kind of HeatTreatment, that processedBy specializes underwentProcess, and which classes a query term may stand for. The ontology is the schema through which the agent sees the graph. It is also much smaller than the graph, curated by few people, and edited over time, which makes it an attractive target. An edit that moves one subclass link changes the answers to every query that passes through it, while every fact in the KG stays intact, so the damage is hard to trace from the answers alone.

Ontology evaluation has long scored the ontology as an artifact, for pitfalls [12], for coverage of competency questions, or for logical consistency. Recent work moves toward measuring how a known ontology defect propagates into retrieval and reasoning, with the KG and the query workload held fixed. This white paper takes the adversary's view of the same setting and asks two questions that a defect catalog cannot answer.

Attack. Given a query workload, a library of edit operators, and a budget of k actions, which chain of edits degrades the answers most? An attacker who knows the ontology and the workload, or who can read an impact ranking of the ontology's concepts, can aim at the few concepts the workload depends on. The search space is a tree of action sequences in which actions enable or conflict with one another. Harm over this tree is not submodular: a look-alike concept does no harm until the relation that attaches it is added, and that relation cannot be added before the concept exists. Greedy search therefore has no approximation guarantee [11] and stops at a chain whose next step looks useless.

Protection. Suppose the defender can lock M triples, so that any edit removing them is sent to a human reviewer instead of being committed. Review capacity is small, so M is small. The defender does not know which chain the attacker will run, only that it has at most k actions. Which locks leave the attacker's best response least harmful?

Contributions. Figure 1 shows the setting.

(1) *Ontology-grounded queries and their change* (Section 2). We define the answer $Q_{\mathcal{O}}(G)$ of a query over a fixed KG under an ontology, the affected area of an ontology update after incremental graph pattern matching [5], and show that a subclass edit can change only a computable region of the answers.

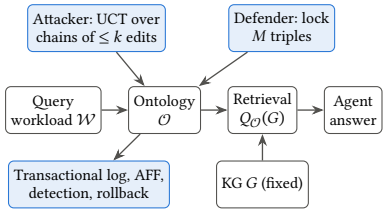


Figure 1: The setting. The ontology decides how the agent retrieves from a fixed KG. Aegis searches for the most harmful edit chains, chooses which triples to lock against them, and logs every edit so that damage is measured, detected, and undone. Blue boxes are this paper's parts.

(2) *Attack search as planning, solved with bandits* (Section 3). We define feasible attack chains under a budget, prove that harm is not submodular, and search the chain tree with UCT [7], which treats each node's next action as a multi-armed bandit and spends its evaluations where harm was found.
(3) *Protection by locking* (Section 4). We pose the choice of M locks as a leader–follower game and lock triples one at a time against the attacker's current best chain, in the manner of the double-oracle method [9].
(4) *A transactional substrate and measurements* (Sections 5 and 6). Aegis applies every edit as a logged transaction with its undo, attributes harm to each action, detects the first harmful commit, and rolls back. We report attack and protection results on four real ontologies, and on a language-model agent that answers in natural language.

2 Ontology-Grounded Queries and Their Change

Ontology and KG. An ontology $\mathcal{O} = (C, R)$ has a set of classes C and a set of triples $R \subseteq C \times P \times C$ over properties P, including subClassOf, written $A \sqsubseteq B$. We write $down_{\mathcal{O}}(X)$ for X and its transitive subclasses and $up_{\mathcal{O}}(X)$ for X and its transitive superclasses. A KG $G = (V, E, \tau)$ has entities V, labeled edges $E \subseteq V \times P \times V$, and asserted types $\tau(v) \subseteq C$. Throughout, the KG is fixed and only the ontology changes.

Querying a KG with an ontology. A query is a graph pattern $Q = (V_Q, E_Q, \ell)$ whose nodes carry class labels and whose edges carry properties. Under $\mathcal{O}$, an entity v matches a pattern node u when one of its types falls under u's class, $\tau(v) \cap down_{\mathcal{O}}(\ell(u)) \neq \emptyset$, and a KG edge labeled p matches a pattern edge labeled p' when p is p' or one of its subproperties. The answer $Q_{\mathcal{O}}(G)$ is the set of matches. The ontology is thus the matching semantics: the same pattern on the same KG has different answers under different ontologies. Ontology-based entity matching [8] and ontology-based subgraph querying [15] relax label equality further, to ontological closeness above a threshold; everything below holds for any semantics in which an ontology edit changes match sets only near the edit.

Quality and harm. A workload $\mathcal{W}$ is a set of queries, each with a gold answer. $P_q(\mathcal{O})$ is the answer F_1 of q under $\mathcal{O}$ against its gold. The *harm* of moving from $\mathcal{O}$ to $\mathcal{O}'$ is the mean drop over the workload,

$$H(\mathcal{O}, \mathcal{O}') = \frac{1}{|\mathcal{W}|} \sum_{q \in \mathcal{W}} \big(P_q(\mathcal{O}) - P_q(\mathcal{O}')\big). \quad (1)$$

Harm is measured over the workload, not one query, so an edit that breaks one answer while repairing another is not counted as harmless.

Affected area. Following incremental graph pattern matching [5], where an update ΔG to the graph changes the match set by AFF, we keep G fixed and update the ontology. For an ontology update $\Delta\mathcal{O}$ from $\mathcal{O}$ to $\mathcal{O}'$,

$$\mathsf{AFF}_q(\Delta\mathcal{O}) = Q_{\mathcal{O}}(G) \triangle Q_{\mathcal{O}'}(G),$$

the matches added or removed. An incremental algorithm is *bounded* when its cost depends only on $|\Delta\mathcal{O}| + |\mathsf{AFF}|$, not on $|G|$.

Proposition 2.1 (Locality of a subclass edit). *Let $\Delta\mathcal{O}$ insert or delete one triple $A \sqsubseteq B$, and let $\mathcal{O}^+$ be whichever of $\mathcal{O}, \mathcal{O}'$ contains it. For a pattern node u, the set of classes under $\ell(u)$ changes only if $\ell(u) \in up_{\mathcal{O}^+}(B)$, and then only by classes in $down_{\mathcal{O}^+}(A)$. Hence the entities whose match status for u changes are typed in $down_{\mathcal{O}^+}(A)$.*

Proof sketch. A class X is under $\ell(u)$ in $\mathcal{O}^+$ but not in the other ontology only if every subclass path from X to $\ell(u)$ in $\mathcal{O}^+$ uses the edge $A \sqsubseteq B$. Such a path runs from X to A, which puts X in $down_{\mathcal{O}^+}(A)$, and from B to $\ell(u)$, which puts $\ell(u)$ in $up_{\mathcal{O}^+}(B)$. □

For single-node queries, Proposition 2.1 gives a maintenance procedure whose cost is $O(|up(B)| + |down(A)| + |\mathsf{AFF}|)$ in the size of the ontology closure and the change, independent of $|G|$ once entities are indexed by type. Aegis's detector uses it to re-score only the queries an edit can reach (Section 5). For multi-node patterns, a changed candidate set propagates through the pattern's joins; for graph updates such propagation is unbounded in general [5], and characterizing the bounded cases for ontology updates is planned work.

The same locality explains why ontology edits amplify. Removing one asserted subclass triple $A \sqsubseteq B$ can remove only inferred triples $X \sqsubseteq Y$ with X in $down(A)$ and Y in $up(B)$, and in a tree-shaped hierarchy it removes all of them, so deep hierarchies turn a small edit into a large change in the inferred closure (Table 1).

3 Attack Search as Planning

Operators. An attacker edits the ontology with four primitive actions: remove a class remove(c) (with every triple touching it), remove a triple remove(r), inject a class inject(c'), and inject a triple inject(r') between existing classes. Each action has a precondition (its target exists, or does not yet exist) and a cost, which we set to 1. Attackers use primitives in recognizable units: a *removal*; a *rewire*, remove($A \sqsubseteq B$) then inject($A \sqsubseteq B'$), which moves a class under a wrong parent; and a *flood*, which injects look-alike classes (Metall beside Metal) and attaches them to the hierarchy.

Definition 3.1 (Feasible attack). A chain $\pi = a_1 \cdots a_n$ is *feasible* under budget k if each a_i is applicable in the state $\mathcal{O} \circ a_1 \cdots a_{i-1}$, its total cost is at most k, it changes at least one triple, and the result $\mathcal{O} \circ \pi$ satisfies $H(\mathcal{O}, \mathcal{O} \circ \pi) > 0$.

Problem 1 (most harmful attack). Given $\mathcal{O}$, a workload $\mathcal{W}$, an operator library, and a budget k, find a feasible chain maximizing $H(\mathcal{O}, \mathcal{O} \circ \pi)$. The companion problem, the cheapest chain whose harm reaches a threshold θ, defines a per-workload *robustness radius*: the fewest edits that make the agent answer wrongly. It is answered by solving Problem 1 for increasing k.

What the attacker knows. A *white-box* attacker knows $\mathcal{O}$, $\mathcal{W}$, and the gold answers, so it can compute H exactly for any candidate. A *gray-box* attacker knows $\mathcal{O}$ and $\mathcal{W}$ and reads an impact ranking of classes, such as the counterfactual Δ_c of Section 5 or an ontology-assessment report; such a ranking is a defender's diagnostic and an attacker's map at once. A *black-box* attacker can only send edits and observe the agent's answers, so each evaluation of H is a costly, possibly noisy query. In every case the scarce resource is the number of *harm evaluations*, and we compare search methods at equal numbers of them.

Proposition 3.2 (Harm is not submodular). *Let $h(S)$ be the harm of applying a set S of actions in a valid order, with an action that cannot be applied left out. There is an ontology, a workload, and actions x, y with $h(\{x, y\}) - h(\{y\}) > h(\{x\}) - h(\emptyset)$.*

Proof. Let $\mathcal{O}$ have Metal $\sqsubseteq$ Material, let the workload ask for the kinds of Material, and let y = inject(Metall) and x = inject(Metall $\sqsubseteq$ Material). Alone, x cannot be applied, so $h(\{x\}) = 0$; alone, y adds an unattached class and changes no answer, so $h(\{y\}) = 0$. Together they put a wrong class into the answer and lower its precision, so $h(\{x, y\}) > 0$, and the marginal gain of x grows when y is added. □

The guarantee of greedy selection for monotone submodular objectives [11] therefore does not apply, and greedy search stops as soon as no single next unit raises harm, even when a pair would. Flooding is exactly such a pair.

3.1 Bandit Tree Search

The chains form a tree: the root is the honest ontology, and a node's children are the units applicable in its state that fit the remaining budget. A depth-first or beam search over this tree spends evaluations uniformly or by a fixed rule. We instead treat the choice of the next unit at every node as a multi-armed bandit, whose arms are the children and whose reward is the harm found below them, and search the tree with UCT [2, 7].

Each iteration of Algorithm 1 has four steps. *Selection* descends from the root by the UCB1 rule [1], choosing the child j of a node visited N times that maximizes

$$\bar{h}_j + c\sqrt{\ln N / n_j}, \tag{2}$$

where $\bar{h}_j$ is the mean reward of child j and n_j its visit count: children that have led to harmful chains are exploited, and rarely tried ones keep an exploration bonus. *Expansion* adds the node's next untried unit, in the order of the impact ranking, so the ranking acts as a prior. Plain UCT expands every child of a node before it descends, and a root with dozens of applicable units would spend the whole budget at depth one. We therefore use *progressive widening* [2]: a node visited N times may have at most $\lceil N^{1/2} \rceil$ children, so the search follows the best-ranked units deep before it widens.

Algorithm 1: UCT attack search

Input: ontology $\mathcal{O}$, harm H, units U ranked by impact, budget k, evaluations E
Output: the most harmful chain scored

root ← node for the empty chain; best ← $(\emptyset, 0)$;
while *fewer than E distinct states scored* **do**
  n ← root; path ← $[n]$;
  while *n has no untried unit, or has $\geq \lceil N_n^{1/2} \rceil$ children* **do**
    n ← unexhausted child of n maximizing Eq. (2); append n to path;
  if *n has an untried unit u that applies* **then**
    n' ← child for $n.\pi \cdot u$, scored by H; append n' to path;
    ρ ← $n'.\pi$ extended by random units up to cost k, scored by H;
  r ← largest harm among path and ρ; update best;
  for *each node on path* **do** add 1 visit and reward r;
return best;

A *rollout* finishes the chain with random applicable units. *Backup* adds the iteration's reward to every node on the path. The reward is the largest harm on the path, not the harm at its end, because the attacker may stop at any prefix.

Properties. Harm values are cached by the digest of the resulting ontology, so two orders that reach the same state cost one evaluation. UCT is *anytime*: it returns the best chain scored so far at any evaluation budget. It is also *complete*: a subtree whose chains have all been expanded is marked exhausted and skipped by selection, every iteration expands at least one unit, and the tree is finite, so with a large enough budget UCT scores every chain and returns the optimum. For bounded rewards, the probability that UCT prefers a suboptimal action at the root converges to zero as iterations grow [7]. Greedy search has neither property here.

Noisy harm (planned). Against a black-box agent, harm is an estimate from sampled answers. The last step of the search then becomes best-arm identification among the top candidate chains: successive elimination or halving returns, with probability at least $1 - \delta$, a chain within ε of the most harmful one, using a number of agent calls that grows with $\log(1/\delta)$ and the inverse squared gaps between candidates [4, 6]. This gives the statistical guarantee that greedy and random search lack; implementing and testing it against the language-model agent is planned.

4 Protection by Locking

A defender who expects an attack cannot review every edit, but can *lock* a few triples. An edit that would remove a locked triple, remove(r) for $r \in L$ or remove(c) for a class c in a locked triple, is not committed; it is queued for a human reviewer. Injections stay open, since a triple cannot be locked before it exists. Review capacity bounds the number of locks by M.

Problem 2 (protection). Given $\mathcal{O}$, $\mathcal{W}$, and budgets k and M, find

$$L^* = \arg\min_{|L| \leq M} \max_{\pi \text{ feasible, cost} \leq k, \pi \text{ avoids } L} H(\mathcal{O}, \mathcal{O} \circ \pi). \tag{3}$$

The inner maximum is the *residual harm* res(L): the harm of the attacker's best response once the locks are in place. The defender moves first and the attacker observes the locks, a leader–follower

Algorithm 2: Best-response locking

Input: ontology $\mathcal{O}$, attacker search $\mathcal{S}$, units U, locks M, candidates c
Output: lock set L

$L \leftarrow \emptyset$;
for $i = 1, \ldots, M$ **do**
  $\pi \leftarrow \mathcal{S}(U \text{ avoiding } L)$; `// attacker's best response`
  if $H(\pi) = 0$ **then return** L;
  $T \leftarrow$ up to c triples removed by π, largest Δ_r first;
  $L \leftarrow L \cup \{\arg\min_{t \in T} \text{res}(L \cup \{t\})\}$;
return L;

(Stackelberg) game as in security games [13], and removing triples so as to limit an adversary's best outcome is a form of network interdiction [14].

A lock stops a chain only if the chain removes it, so the defender's choice acts on the removal sets of harmful chains. When those chains are listed explicitly, deciding whether M locks bring the residual harm below a threshold θ is the Hitting Set problem over the chains with harm above θ, and is NP-hard. Whether it stays hard when the chains are induced by an ontology and a workload is open.

Three lock strategies. *Top impact* locks the M triples with the largest counterfactual impact Δ_r, the harm of removing each one alone; it is cheap but blind to how actions combine and to which substitutes the attacker has. *Random* locks M triples drawn from those with $\Delta_r > 0$, a floor. *Best response* (Algorithm 2) adds one lock per round: it runs the attacker against the current locks, and among the triples removed by the attacker's best chain it locks the one that leaves the smallest residual harm. Like the double-oracle method [9], it grows the defender's strategy only in answer to attacks the attacker would actually run, instead of enumerating every chain.

The attacker oracle matters. Residual harm is only as trustworthy as the attacker search that computes it. A weak attacker understates the threat, and a lock set can then look harmful to the defender: in our runs, locking triples on one ontology steered beam search toward a chain more harmful than the one it had found without locks, so the "residual" rose. We therefore evaluate every lock set against the best of beam search and UCT (Section 6). Randomized lock sets, which make the defender harder to anticipate, and a queueing model of the reviewer are planned.

5 The Aegis Substrate

Attack and protection both need a store that can apply an edit, measure it, and take it back. Aegis treats the ontology the way a database treats its data.

Transactions. Each primitive action is applied atomically with its undo actions, and a chain is all or nothing. Every commit is logged with digests of the state before and after, so any past state can be rebuilt and any two orders that reach the same state are recognized. The log runs in memory, in a JSON-lines file, or in an RDF database with a write-ahead log, where each action is a SPARQL Update transaction and a rollback arrives as compensating transactions, in the manner of ARIES [10].

Impact and attribution. The counterfactual impact of a class is $\Delta_c = P(\mathcal{O}) - P(\mathcal{O} \setminus c)$, and Δ_r likewise for a triple. Computing Δ_c for every class costs one evaluation per class; group testing over subtrees finds the same values with fewer evaluations when few classes matter. For a chain, Aegis attributes the affected area to each action given the ones before it, and minimizes the chain to its net effect.

Detection and recovery. Health checks (acyclic hierarchy, domain and range, disjointness, and an OWL reasoner's satisfiability) run on every commit. A detector re-scores only the queries a commit can reach (Proposition 2.1), alarms when harm passes a threshold, and rolls back to the last commit before the alarm, keeping every honest edit made earlier. A decision layer either rejects a pending chain, repairs committed damage by compensating the fewest commits that explain it, or rolls back when repair cannot.

Agents. Questions in natural language are answered by a language model that writes SPARQL against a labeled schema of the ontology; the answer is scored like any other. The same attack, attribution, and repair machinery applies to an agent's shared memory, where an attacker forges or removes stored findings.

6 Experiments

We ask three questions. **[Q1] Attack**: at the same number of harm evaluations, which search finds the most harmful chains? **[Q2] Protection**: how much residual harm do M locks leave, and does it matter how they are chosen? **[Q3] Substrate**: is the damage detected and undone, and does a language-model agent suffer it as the template answerer predicts? Every number below comes from the scripts in `experiments/`, and the figures are built from their result files.

6.1 Setup

Ontologies. Three materials-science ontologies (EBSD, MatProc, XRD) and the OAEI mouse anatomy ontology (Table 1).

Workload and harm. For each ontology, the five classes with the most subclasses become questions "What kinds of X are there?", whose gold answer is the set of X's transitive subclasses in the honest ontology. Each answer is scored by F_1 and harm is Eq. (1). These are single-node queries over the ontology itself; queries over a KG are planned (Section 8).

Units and searches. Units are removals of the 8 classes and 8 subclass triples with the largest impact, and rewires of those triples to the 5 most depended-on classes (44 to 49 units). Greedy and beam (width 3) run to completion, and beam's evaluation count is the budget given to random search and UCT, each over 5 seeds. UCT uses $c = 0.5$ and widening exponent $1/2$; we chose the exponent from $\{0.3, 0.5, 0.7\}$ on EBSD, MatProc, and XRD, so results there are tuned and mouse anatomy is held out.

Protection. Locks are chosen among the subclass triples the workload reaches, against a $k = 2$ attacker. Residual harm is the best of beam and UCT at 300 evaluations. Random locking averages 5 seeds.

Table 1: Ontologies. Calls: evaluations to compute every class's Δ_c, by group testing / exactly. Harm: the worst two-action chain. Ampl.: inferred subclass triples it changes per asserted one.

Ontology	Classes	Calls	Harm, k=2	Ampl.
EBSD	54	55 / 55	0.400	0.50
MatProc	127	253 / 128	0.800	2.09
XRD	354	445 / 355	0.400	2.01
Mouse anatomy	2,743	**2,227** / 2,744	0.745	**5.80**

Cost. No experiment in this section calls a model except the language-model agent, whose 20 requests cost $0.68 at list prices. All other runs take seconds to minutes on a laptop.

6.2 Q1: Attack Search

Figure 2 compares the four searches at equal cost. At $k = 1$ every search finds the same removal. Beyond it, the methods split by ontology. On XRD, UCT finds chains of harm 0.400, 0.600, and 0.763 for $k = 2, 3, 4$, where greedy and beam both stop at 0.244, 0.444, and 0.644: 64%, 35%, and 18% more harm at the same cost. UCT's chains remove one subclass link and then whole upper-level classes, and each action raises the harm by exactly 0.2, the weight of one of the five queries: every action empties a different query's answer. On MatProc, EBSD, and the held-out mouse anatomy, UCT ties beam at $k = 2$ and trails it at $k \geq 3$ (0.884 against 0.945 on mouse anatomy at $k = 3$): when the harm of a chain is close to the sum of its units' harms, a narrow beam that keeps the best partial chains is hard to beat at a small budget. Greedy is never better than beam, and on MatProc it finds about half of beam's harm at $k = 2$ (0.419 against 0.800, where the best pair removes two classes): committing to one first step loses the pair that beam and UCT keep among their alternatives.

Figure 3 shows the anytime behavior. Given more evaluations, UCT keeps improving and settles: at 200 evaluations it reaches 1.000 on MatProc and 0.945 on mouse anatomy, matching beam's best, with a standard deviation of zero over seeds, while random search still varies by 0.04 to 0.12 between seeds. Random search at a small budget sometimes beats UCT (EBSD at 200 evaluations, 0.533 against 0.493), since progressive widening commits early to the units the impact ranking prefers. The two searches differ in what they guarantee: UCT is complete and converges, random search is neither.

6.3 Q2: Protection

Figure 4 shows the residual harm after M locks. Best-response locking is never worse than locking the highest-impact triples, and is much better where the attacker has substitutes. On EBSD, six top-impact locks leave the residual harm at 0.400, unchanged from no locks, because the attacker moves to the next query's triples; six best-response locks bring it to 0.133. On XRD the residual is 0.030 against 0.212. On MatProc and mouse anatomy the highest-impact triples are also the ones the attacker needs, and the two strategies agree (0.025 and 0.019–0.029 at $M = 6$). Random locks of relevant triples protect little: on mouse anatomy, ten of them leave 0.569 of the 0.745 harm. A handful of well-chosen locks, six to ten triples out of hundreds or thousands, removes nearly all of a two-action attacker's harm on every ontology.

Table 2: A language-model agent under the worst two-action chain.

Ontology	Honest	Harm, template	Harm, agent	USD
EBSD	1.000	0.400	0.400	0.034
MatProc	1.000	0.800	0.800	0.048
XRD	1.000	0.400	0.400	0.146
Mouse anatomy	1.000	0.745	0.745	0.454

6.4 Q3: Detection, Recovery, and a Language-Model Agent

On each ontology, the worst two-action chain was applied under the detector with a threshold of 0.05: every attack was detected at its commit and rolled back to the exact honest state. To check that harm on the template answerer is the harm an agent suffers, we asked the same workloads in natural language ("What kinds of venous blood vessel are there?"), answered by Claude Opus 5.5 writing SPARQL against a labeled schema (Table 2). The agent answered every honest question correctly, mapping each to the right class even among 2,737 mouse anatomy classes named by opaque identifiers, and then suffered exactly the template answerer's harm under attack. For hierarchy questions the damage lies in the ontology, not in how the question is read, so the cheap answerer is a faithful proxy during attack search.

The same substrate defends agent memory. Over 20 random attacks on a shared memory store, each forging or removing one to three findings between honest work, every attack was detected; repair by attribution restored every probe answer and kept all honest work done after the attack, where rollback restored the probes and lost that work.

7 Related Work

Ontology evaluation. Pitfall scanners [12] and competency-question coverage score an ontology against modeling rules or the vocabulary a task needs. They measure a given ontology; Aegis asks how far a bounded adversary can move those measures, and which triples to protect.

Ontology-based matching and incremental matching. Ontology-based subgraph querying [15] and entity matching [8] use ontological closeness to decide matches, and incremental graph pattern matching [5] defines the affected area and boundedness for graph updates. We carry the affected area to ontology updates over a fixed graph. An edit to one ontology also changes its alignment to another, a case we leave to future work.

Poisoning knowledge and retrieval. Data poisoning of KG embeddings [16] perturbs facts to change link predictions, and PoisonedRAG [17] injects passages to steer a retrieval-augmented model. Both edit the data. Here the data is fixed and the schema that interprets it is attacked, which is why the damage reaches answers indirectly and why its affected area can be computed from the ontology alone.

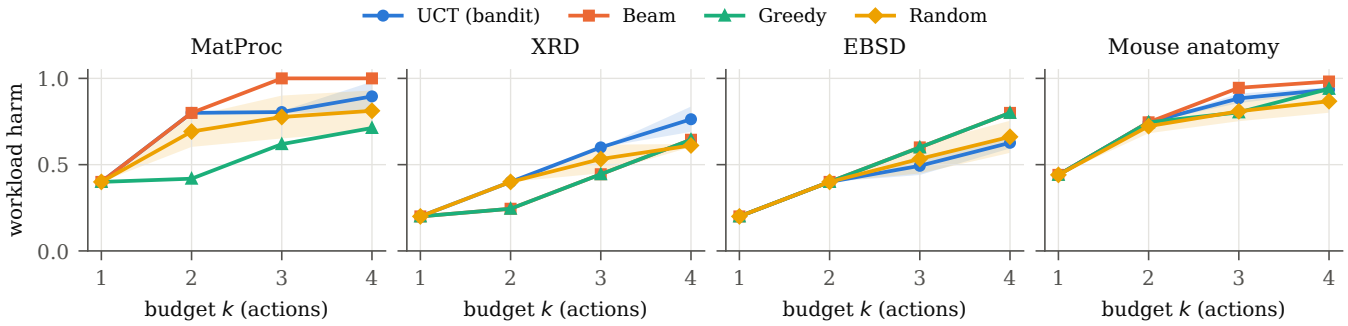


Figure 2: Most harmful chain found within a budget of k actions, at equal harm evaluations (beam's count). Bands: one standard deviation over 5 seeds.

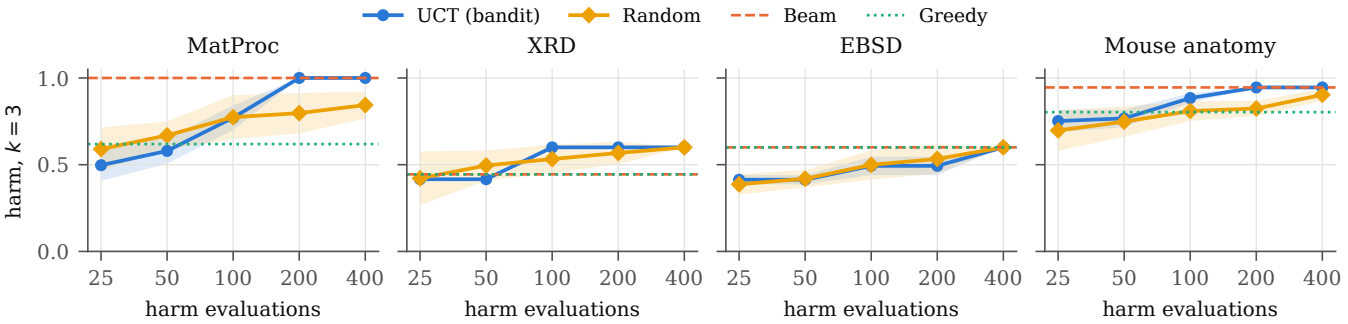


Figure 3: Harm found against harm evaluations, $k = 3$. Dashed and dotted lines: beam and greedy at their own evaluation counts.

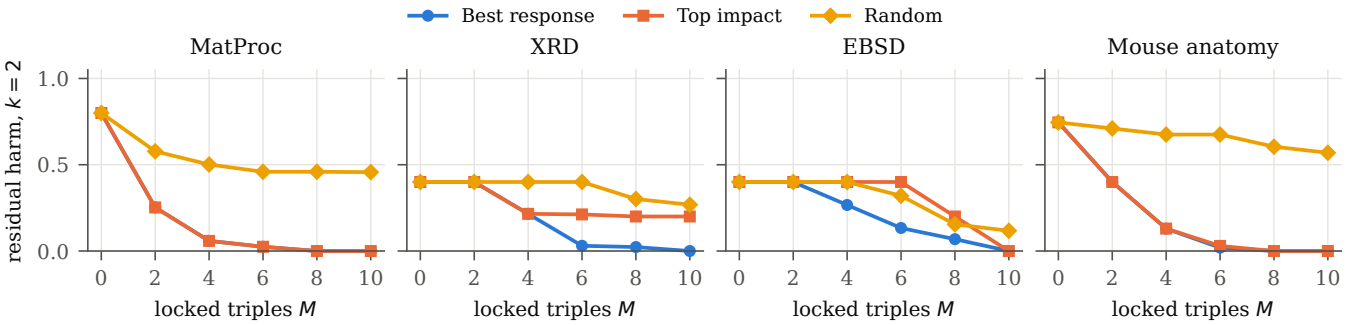


Figure 4: Residual harm of a $k = 2$ attacker after M locks, against the best of beam and UCT.

Search and games. UCT [2, 7] builds on the UCB1 bandit rule [1]; best-arm identification [4, 6] gives PAC guarantees under noise. Our protection problem is a Stackelberg game [13] and an interdiction problem [14], and best-response locking follows the double-oracle idea [9].

8 Outlook

This white paper states two adversarial problems on ontology-grounded retrieval and gives a first measured answer to each: bandit tree search finds harmful chains that greedy search cannot reach, and locking triples in answer to the attacker's best response protects more than locking the highest-impact ones. The next steps, in order:

(1) *Ontologies over real KGs.* The experiments use class-level queries on the ontology itself. Next is pattern queries over a materials KG, with gold answers from domain experts, so that $Q_{\mathcal{O}}(G)$ and its affected area are measured on entities.

(2) *Bounded maintenance.* Extend Proposition 2.1 from single-node queries to patterns, and identify which ontology updates admit bounded incremental maintenance.

(3) *Noisy harm.* Replace exact harm with sampled answers from a language-model agent and add best-arm identification to the search, with its (ε, δ) guarantee.

(4) *Richer operators and costs.* Equivalence, disjointness, domain and range, and property-chain edits, with costs that reflect how conspicuous each edit is to a curator.
(5) *Repair under a budget.* After an attack lands, choose which edits to undo or which triples to re-assert with a limited budget of reviewer time, as an ontology-cleaning problem.

References

[1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 2–3 (2002), 235–256.

[2] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 1–43.

[3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130

[4] Eyal Even-Dar, Shie Mannor, and Yishay Mansour. 2006. Action Elimination and Stopping Conditions for the Multi-Armed Bandit and Reinforcement Learning Problems. *Journal of Machine Learning Research* 7 (2006), 1079–1105.

[5] Wenfei Fan, Jianzhong Li, Jizhou Luo, Zijing Tan, Xin Wang, and Yinghui Wu. 2011. Incremental Graph Pattern Matching. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 925–936.

[6] Zohar Karnin, Tomer Koren, and Oren Somekh. 2013. Almost Optimal Exploration in Multi-Armed Bandits. In *International Conference on Machine Learning (ICML)*. 1238–1246.

[7] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *European Conference on Machine Learning (ECML)*. 282–293.

[8] Hanchao Ma, Morteza Alipourlangouri, Yinghui Wu, Fei Chiang, and Jiaxing Pi. 2019. Ontology-based Entity Matching in Attributed Graphs. *Proceedings of the VLDB Endowment* 12, 10 (2019), 1195–1207.

[9] H. Brendan McMahan, Geoffrey J. Gordon, and Avrim Blum. 2003. Planning in the Presence of Cost Functions Controlled by an Adversary. In *International Conference on Machine Learning (ICML)*. 536–543.

[10] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems* 17, 1 (1992), 94–162.

[11] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions. *Mathematical Programming* 14, 1 (1978), 265–294.

[12] María Poveda-Villalón, Asunción Gómez-Pérez, and Mari Carmen Suárez-Figueroa. 2014. OOPS! (OntOlogy Pitfall Scanner!): An On-line Tool for Ontology Evaluation. *International Journal on Semantic Web and Information Systems* 10, 2 (2014), 7–34.

[13] Milind Tambe. 2011. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned.* Cambridge University Press.

[14] R. Kevin Wood. 1993. Deterministic Network Interdiction. *Mathematical and Computer Modelling* 17, 2 (1993), 1–18.

[15] Yinghui Wu, Shengqi Yang, and Xifeng Yan. 2013. Ontology-based Subgraph Querying. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 697–708.

[16] Hengtong Zhang, Tianhang Zheng, Jing Gao, Chenglin Miao, Lu Su, Yaliang Li, and Kui Ren. 2019. Data Poisoning Attack against Knowledge Graph Embedding. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 4853–4859.

[17] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *USENIX Security Symposium*.